

Performance Testing

Date	29 September 2026
Team ID	6ab2a427dbd693aad5a7d4b
Project Name	Student Enrollment & Course Management System (SECMS)
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	JMeter, Locust, Postman - JMeter for load testing, Locust for 1000+ virtual users simulation, Postman for API endpoint testing
Type of Testing	Load Testing, Stress Testing, Spike Testing - Load test for normal admission rush, Stress test for 2x peak load, Spike for sudden login surge
Target Module / API	Student Registration API (/api/register), Course Catalog API (/api/courses), Enrollment API (/api/enroll with seat lock), Payment API (/api/pay), Admin Report API (/api/reports)
Test Environment	Local + Cloud Hosted Server - Windows 11, Node.js backend, MySQL RDS, Deployed on Render/AWS with 2GB RAM instance
Test Date	28 September 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Student Course Browse & Search - 100 students viewing course catalog simultaneously with filter	100	60	Response time < 2 sec, No error, All courses load with seat count
2	Concurrent Enrollment Stress	500	120	No double booking, Only 50

	Test - 500 students trying to enroll same popular course with 50 seats (check seat locking)			success, 450 fail with proper message, ACID maintained
3	Payment & Report Generation Load - 200 students doing fee payment via Razorpay mock + Admin generating PDF reports	200	90	Payment API < 3 sec, Report generation < 5 sec, 0% transaction loss

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.34 seconds	Pass	Fast with Redis cache for course list, DB indexing works well
2	Response Time (Max)	< 5 seconds	3.8 seconds	Pass	Max during enrollment peak, still under target due to seat locking queue
3	Throughput (Request)	> 100 req/sec	145 req/sec	Pass	Handled 145 requests per second during 500 user test
4	Error Rate	< 1%	0.4%	Pass	Only 2 errors out of 500 during stress - due to network timeout not logic
5	CPU Utilization	< 80%	68%	Pass	Stable even at 500 users, Node.js clustering helped
6	Memory Utilization	< 80%	72%	Pass	MySQL connection pooling optimized, no memory leak

Step 4: Observations & Analysis

Aspect	Details
Key Findings	System handled 500 concurrent enrollments without double booking. Avg response 1.34 sec, throughput 145 req/sec. Seat locking logic 100% accurate. Payment flow stable
Bottlenecks Identified	Initial bottleneck at MySQL without indexing - course query took 4 sec. Fixed with indexing on course_id and student_id. Second bottleneck at file upload - moved to S3 async
Optimization Steps Taken	Added Redis cache for seat availability, MySQL indexing, PM2 cluster mode for Node.js, Connection pooling (limit 20), Nginx reverse proxy, S3 for document storage, Rate limiter to prevent spam

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



